Interval DP & Digit DP

DP over segments [l, r], and counting numbers digit by digit

Learning goals

- Recognize **interval DP**: the state is a contiguous segment `[l, r]`
- Choose between "**last** operation splits" and "**first/ends**" transitions
- Fill tables in the right order (**by length**, or `l` descending)
- Solve merging (*Slimes*), games (*Removal Game*), and string-removal problems
- Count integers in `[A, B]` with a digit property using **digit DP**
- Handle the **tight** flag, **leading zeros**, and `f(B) − f(A−1)`

Part 1 — Interval DP

Typical signals:

- A **sequence** where operations act on **adjacent** elements or on the **ends**
- After an operation, the problem splits into **independent sub-segments**
- $n \le 500$ ($O(n^3)$) or $n \le 5000$ ($O(n^2)$)

State: `dp[l][r]` = answer for the subarray `a[l..r]`.

Motivating problem — AtCoder DP N *Slimes*

n ≤ 400 slimes in a row with sizes `a[i]`. Repeatedly merge two **adjacent** slimes; cost = sum of sizes. Minimum total cost to get one slime?

```
10 20 30 40
(10+20)=30          cost 30
(30+30)=60          cost 60
(60+40)=100         cost 100   total 190 ✔ optimal
```

Greedy "merge the cheapest adjacent pair" fails: `2 2 1 2` → greedy 3+5+7 = 15, optimal (2+2), (1+2), (4+3) = 14.

Think about the last merge

The final merge joins `[l..k]` and `[k+1..r]` for some split k.

- Both halves were built **independently** before
- Last merge costs `sum(l..r)` regardless of k

```
dp[l][r] = min over l ≤ k < r of (dp[l][k] + dp[k+1][r]) + sum(l, r)
dp[i][i] = 0
```

States O(n²), transitions O(n) → **O(n³)** = 6.4·10^7 for n = 400 ✔

Filling order

`dp[l][r]` needs **shorter** segments → iterate by length:

```
for (int len = 2; len <= n; len++)
    for (int l = 0; l + len - 1 < n; l++) {
        int r = l + len - 1;
        dp[l][r] = INF;
        for (int k = l; k < r; k++)
            dp[l][r] = min(dp[l][r], dp[l][k] + dp[k+1][r]);
        dp[l][r] += pre[r + 1] - pre[l];
    }
```

Alternative: `l` from n−1 down to 0, `r` from l up to n−1. Or memoized recursion.

Table picture

```
        r=0   r=1   r=2   r=3
l=0      0    30    90   190
l=1           0     50   140
l=2                 0     70
l=3                       0
```

(a = 10 20 30 40) — fill diagonals outward from the main one; answer top-right.

Removal game — CSES 1097

Numbers in a row; two players alternately take the **first or last**. Both play optimally to maximize their own sum. First player's score?

State: `dp[l][r]` = (current player's total) − (opponent's total) on `x[l..r]`

```
dp[l][l] = x[l]
dp[l][r] = max(x[l] − dp[l+1][r],  x[r] − dp[l][r−1])
```

Answer: first = (total + dp[0][n−1]) / 2.

Why the difference trick?

- Zero-sum structure: after I take x[l], the opponent faces `[l+1, r]` and is now "the current player"
- Their advantage `dp[l+1][r]` is **my** disadvantage
- One number per state instead of two
- `4 5 1 3` → dp = 3, first player gets (13 + 3)/2 = **8** ✔

Pattern: "first element's partner"

Problems like *Clear the String* (CF 1132F): delete blocks of equal letters; cost 1 per deletion. Minimum deletions?

```
dp[l][r] = 1 + dp[l+1][r]                              // delete s[l] alone
dp[l][r] = min(dp[l][r], dp[l+1][k−1] + dp[k][r])  for s[k] == s[l]
```

Idea: `s[l]` is deleted **together with** some later equal `s[k]` — clear the middle first.

Interval DP patterns cheat sheet

Pattern	Transition	Examples
last split	`dp[l][k] ⊕ dp[k+1][r]`	Slimes, matrix chain
take ends	from `dp[l+1][r]`, `dp[l][r−1]`	Removal Game, AtCoder DP L *Deque*
match `l` with `k`	`dp[l+1][k−1] ⊕ dp[k][r]`	Clear the String, Zuma
palindromes	`s[l]==s[r]` → `dp[l+1][r−1]`	longest palindromic subsequence

Knuth's optimization (Nationals) speeds some $O(n^3)$ cases to $O(n^2)$.

Part 2 — Digit DP

Count integers x in [A, B] with property P(x), where B up to 10^18 (or 10^10000 as a string).

- Can't iterate over numbers
- But P usually depends on digits **left to right** with a small "memory"
- Count `f(N)` = # of x in [0, N]; answer = `f(B) − f(A − 1)`

Examples: digit sum divisible by D; no two adjacent equal digits; contains "13"; at most 3 non-zero digits.

Building numbers ≤ N digit by digit

Write N = $d_0\ d_1 \ldots d_{L-1}$. Build x position by position.

- **tight** = "so far x's prefix equals N's prefix"
  - tight → next digit ≤ d_pos; stays tight only if equal
  - not tight → any digit 0..9
- Once x is strictly smaller in some position, the rest is **free**

```
N = 3 2 7
x = 3 1 _   ← became free at position 1: last digit any of 0..9
x = 3 2 _   ← still tight: last digit 0..7
```

State

```
go(pos, state, tight, started)
```

- `pos` — current position (0..L)
- `state` — what P needs to remember (digit sum mod D, last digit, ...)
- `tight` — bound by N?
- `started` — have we placed a non-zero digit yet? (leading zeros)

Memoize only the **non-tight** states — tight states form a single path.

Example — CSES 2220 *Counting Numbers*

Count x in [a, b] with **no two adjacent equal digits** (0 ≤ a ≤ b ≤ 10^18).

- state = last digit (or "none" while in leading zeros)
- leading zeros must **not** count as equal digits: `0 0 7` is really `7`

```
long long go(int pos, int last, bool started, bool tight) {
    if (pos == (int)s.size()) return 1;
    if (!tight && seen[pos][last][started]) return memo[pos][last][started];
    int limit = tight ? s[pos] - '0' : 9;
    long long res = 0;
    for (int d = 0; d <= limit; d++) {
        bool nt = tight && d == limit;
        if (!started && d == 0) res += go(pos + 1, 10, false, nt);
        else if (!started || d != last) res += go(pos + 1, d, true, nt);
    }
    if (!tight) { seen[pos][last][started] = true; memo[pos][last][started] = res; }
    return res;
}
```

Checking it

- `f(x)` counts [0, x] — the number 0 is counted once (all leading zeros)
- answer = `f(b) − f(a − 1)`, with `f(−1) = 0`
- Sample: `123 321` → **171** ✔
- Brute-force check on all ranges ≤ 3000 ✔

Complexity: positions 19 × states 11 × 2 × 10 digits — trivial.

Iterative digit DP — AtCoder DP S *Digit Sum*

Count x in [1, K] (K up to 10^10000) with digit sum ≡ 0 (mod D), D ≤ 100.

Keep `dp[r]` = count of **free** prefixes with digit sum ≡ r, and track the single tight prefix separately:

```
for (char ch : K) {
    int c = ch - '0';
    fill(ndp.begin(), ndp.end(), 0);
    for (int r = 0; r < D; r++)                    // free -> free
        for (int d = 0; d <= 9; d++) ndp[(r + d) % D] += dp[r];   // (mod)
    for (int d = 0; d < c; d++) ndp[(tightSum + d) % D] += 1;    // tight -> free
    tightSum = (tightSum + c) % D;
    swap(dp, ndp);
}
// answer = dp[0] + (tightSum == 0) - 1   (K itself; remove x = 0)
```

O(|K| · D · 10) = 10^7 ✔. Leading zeros harmless: they add 0 to the sum.

When are leading zeros a problem?

Property	Leading zeros matter?
digit sum, sum mod D	no (0 adds nothing)
number mod M	no
adjacent digits differ	**yes** — need `started`
count of zero digits	**yes**
"contains digit 0"	**yes**

When in doubt, carry `started`.

Complexity

Problem	States	Transition	Total
Slimes	n^2	n	$O(n^3)$
Removal Game	n^2	1	$O(n^2)$
Digit DP	$L \cdot S \cdot 2$	10	$O(10 \cdot L \cdot S)$

L = number of digits, S = size of the extra state.

Common pitfalls

- Interval DP filled in **row-major** order → reads uncomputed cells
- `dp[l][r]` arrays of `long long` at n = 5000 → 200 MB: use `int` or roll
- Forgetting `+ sum(l, r)` / mixing inclusive and exclusive prefix sums
- Digit DP: memoizing **tight** states (wrong if N changes between calls) — or forgetting to reset memo when N changes
- Off-by-one in `f(A − 1)` and the special case `A = 0`
- Counting 0 when the problem says "positive integers"
- Treating leading zeros as real digits

Practice problems

- AtCoder DP Contest L *Deque*, N *Slimes*, S *Digit Sum*
- CSES 1097 *Removal Game*
- CSES 2220 *Counting Numbers*
- CSES 1744 *Rectangle Cutting* (2D cut DP)
- Codeforces 1132F *Clear the String*
- Codeforces 607B *Zuma*
- Codeforces 1036C *Classy Numbers*
- Codeforces 628D *Magic Numbers*
- AtCoder ABC 154 E *Almost Everywhere Zero*

Summary

- **Interval DP**: state `[l, r]`, think about the last operation or the ends
- Fill by increasing length; typical $O(n^2)$ or $O(n^3)$
- Game on a line → store the **score difference**
- **Digit DP**: count in [0, N] with (pos, state, tight, started)
- Answer = `f(B) − f(A − 1)`; memoize non-tight states
- Next week: strings — hashing, prefix function, Z-function